Trabajo Práctico Integrador (UVA 1 y UVA 2): Arquitectura del Sistema de Control Sky-Delivery S.A.

GRUPO 9

1. Contexto de Negocio: Infraestructura de Logística Autónoma

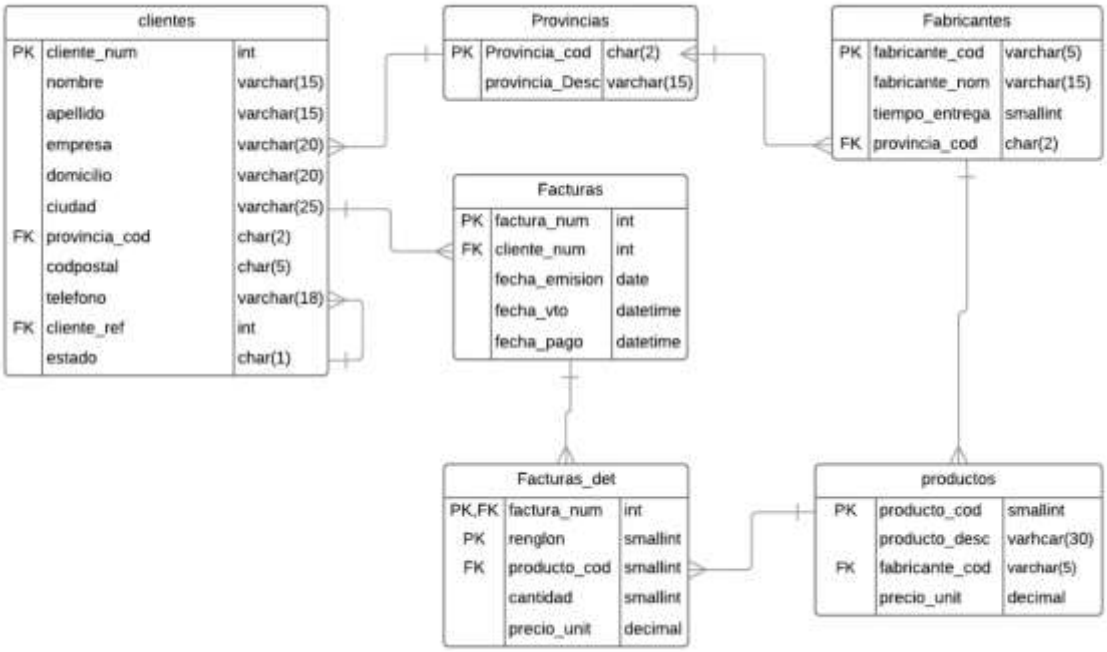
La corporación **Sky-Delivery S.A.** se encuentra en una fase crítica de despliegue tecnológico. Es imperativo que la arquitectura de software que soporte su flota de drones de reparto sea robusta, escalable y esté diseñada bajo el rigor del **Paradigma Orientado a Objetos (POO)** . No se admitirán soluciones basadas en lógica procedimental; se exige un diseño que mitigue la redundancia mediante el encapsulamiento y un desacoplamiento efectivo entre componentes.El sistema debe coordinar de manera precisa las siguientes entidades:

- **Drones:** Unidades autónomas que gestionan internamente niveles de energía y telemetría (coordenadas GPS).
- **Paquetes:** Entidades con dimensiones físicas y peso que condicionan la autonomía del transporte.
- **Centros de Despacho:** Nodos logísticos responsables de la orquestación y asignación de rutas.Como arquitectos, su misión es garantizar que el diseño soporte la complejidad operativa de una flota de alta disponibilidad, asegurando que la transición entre el modelo teórico y la ejecución en la Máquina Virtual sea impecable.

2. Desafío de Modelado UML (UVA 1)

2.1 Identificación de Componentes y los 3 Pilares

3. Desafíos Técnicos de Arquitectura y Memoria (UVA 2)



3.1 Gestión en la Máquina Virtual (VM)

- Segmentacion de memoria: La refernecia del dron vive en el Stack (Memoria temporal). La instancia fisica con sus datos vive en el heap (Memoria dinamica).
  Cuando la variable pierde su alcance (Scope), el Stack destruye la referencia y el objeto en el Heap queda suelto hasta que el Garbage Collector lo elimina para liberar memoria.
-

a) Compilación: De código fuente a Bytecode.
b) Carga y Verificación: Se sube a RAM y se audita por seguridad.
c) Ejecución: El Intérprete lee línea por línea, mientras que el Compilador JIT traduce los fragmentos más repetitivos directamente a código máquina nativo para máxima velocidad.

3.2 Tipos de Datos y Wrappers

Se debe usar el primitivo double debido que es más rápido, ocupa menos memoria y evita conversiones innecesarias (autoboxing/unboxing) que exigen ciclos de CPU. Se usa double solo si necesitas permitir valores nulos (null) o si debes guardarlo en colecciones que no aceptan primitivos (como ArrayList<Double>).

3.3 Estructuras de Control y Manejo de Arreglos

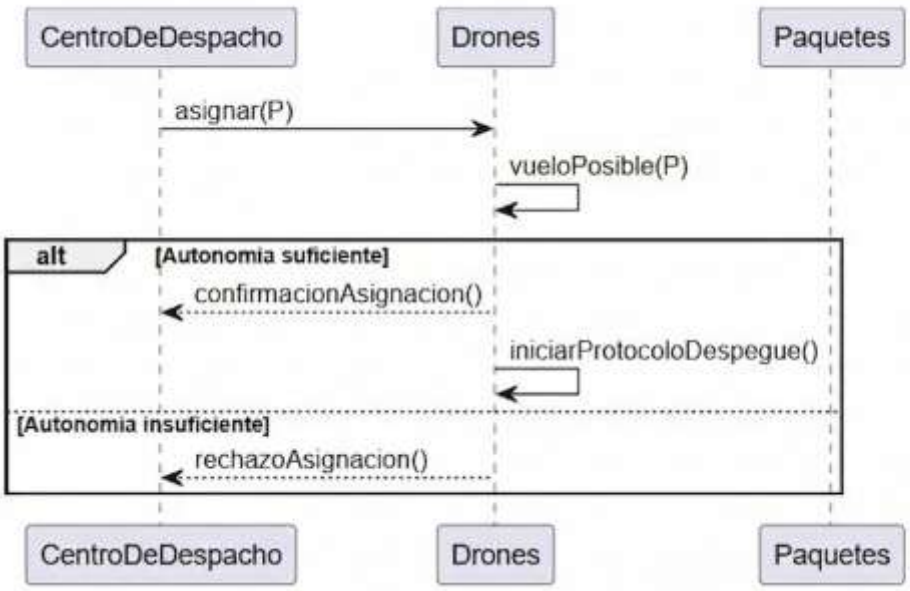
Logica de Autorizacion:

```
public boolean autorizarVuelo() {
   if (this.paquete == null) return false;
   double bateriaRequerida = calcularConsumo(this.paquete.getPeso());
   return this.nivelDeBateria >= bateriaRequerida && this.estadoOperativo;
}
```

Instancia de flota:

```
Drone [] flota = new Drone[50]

for (int i = 0; i < flota.length; i++) {
flota[i] = new Drone();
}
```

Integración Dinámica: Diagrama de Secuencia

4.1 Escenario de Interacción Crítica